Project 4 — StreamBharat: OTT Streaming Analytics Platform

Project brief

You are a data engineer at **StreamBharat**, a video streaming app with 3 crore registered users, a free tier with ads and two paid plans. It streams movies, web series in 8 Indian languages and live cricket. The content team wants to know what people actually watch, finance must pay content partners correctly, the ad-sales team must deliver what advertisers paid for, and growth wants to stop paying subscribers from leaving.

Datasets

Table	Source and format	Key columns
users / profiles	App backend, JSON lines	user_id, signup_ts, city, state, age_band, acquisition_channel / profile_id, user_id, is_kids
subscription_events	Billing service, JSON lines	user_id, event_type (trial_start/subscribe/renew/upgrade/downgrade/pause/resume/cancel/payment_failed), plan, price, event_ts, gateway
gateway_settlements	Payment gateways, CSV	gateway, user_id, amount_inr, settled_ts, event_type
catalog	Content system, nested JSON (monthly snapshots)	content_id, title, type (movie/series/live), languages[], genres[], cast[{name, role}], release_date, age_rating, partner_id, tier; series hold seasons[episodes[content_id, episode_no, duration_sec]]
licences	Content system, CSV	content_id, region (state), licence_start, licence_end
playback_events	Player SDK, JSON lines (one file per server-arrival day)	event_id, user_id, profile_id, device_id, content_id, event_type (start/heartbeat/pause/seek/stop/complete), position_sec, client_ts, server_ts, ip_city, device_type, app_version, download_id
search_logs	Search service, JSON lines	search_id, profile_id, query_text, results_count, clicked_content_id, ts
ad_campaigns / ad_breaks / ad_impressions	Ad server, CSV	campaign_id, budget_inr, dates, daily_frequency_cap, target_genres / break_id,

Table	Source and format	Key columns
		user_id, content_id, break_ts, filled / impression_id, campaign_id, ad_break_ts, completed, cpm_inr
cdn_logs	Two CDN providers, raw text in two formats	time, edge, client hash, content_id, quality, segment, status, bytes, user agent
ratings	App, JSON lines	profile_id, content_id, rating (1–5), ts
content_partners, cdn_rates, cdn_edges, genre_mapping, city_state, plans, published_metrics	Reference CSVs	revenue share and minimum guarantee; rate per GB; edge → region; raw → standard genre; city → state; plan screens; numbers already sent to the board

Dataset: download `StreamBharat_dataset.zip` (about 36 MB). It is a sample of 2,500 accounts: subscriptions from May 2025, playback, search and ads for July – October 2026, CDN logs for September 2026. In this dataset the player sends a heartbeat every **5 minutes**, not every 30 seconds. It includes a README and `ott_stream_simulator.py` for Q17 and Q18.

Folder layout

```
streambharat/
├── data/raw/app/ player/ billing/ ads/ cdn/ content/
├── lake/bronze/ silver/ gold/
├── src/ingest/ transform/ engagement/ revenue/ ads/ recsys/ streaming/ quality/ ml/
├── src/pipeline.py
└── tests/
```

Getting started

Hints point you in the right direction; they never give the answer. Try each question on your own first.

1. Install Python 3.10+ and Java 17, then run `pip install pyspark==3.5.* delta-spark pytest`.
2. Download `StreamBharat_dataset.zip`, unzip it, and put its `data/` folder inside your `streambharat/` project folder. Read the dataset's README first.
3. Create one shared Spark session helper and import it in every module:

```
# src/common/spark_session.py
from pyspark.sql import SparkSession

def get_spark(app="StreamBharat"):
    return (SparkSession.builder.appName(app).master("local[*]")
            .config("spark.sql.shuffle.partitions", "8")
            .config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension")
            .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

4. Work through the questions in order: later questions read the tables earlier ones produce.
5. For Q17 and Q18, run `python ott_stream_simulator.py --stream plays` (or `ads`) in a second terminal while your streaming job runs.
6. Remember: in this dataset one heartbeat stands for up to 5 minutes of watching.

Questions 1–10

Q1. Ingest the nested content catalog

Scenario: The content system exports the catalog as deeply nested JSON: each title has arrays of languages, genres and cast members (with roles), and series contain seasons that contain episodes. Analysts cannot answer simple questions like "How many Marathi comedies do we have?"

Data: catalog JSON.

Tasks

1. Read the catalog with an explicit nested schema.
2. Create a flat `dim_content` with one row per title or episode, including its series and season.
3. Create bridge tables `content_genre`, `content_language` and `content_cast` (one row per pair).
4. Standardise genre names ("Rom-Com", "romcom", "Romantic Comedy") with a mapping table.
5. Count titles and total hours by language and genre.

Expected output: `dim_content` plus three bridge tables.

Concepts tested: nested schemas, `explode` on arrays of structs, many-to-many modelling, mapping tables.

Hint: Series hold seasons, and seasons hold episodes — that is two levels of arrays to explode. `reference/genre_mapping.csv` covers the genre spellings.

Q2. Clean playback events and fix device clocks

Scenario: Player events carry two timestamps: the phone's clock (`client_ts`) and the server's (`server_ts`). Some phones are set hours or even years wrong, and offline downloads upload their events days later. Watch-time reports are off as a result.

Data: playback_events.

Tasks

1. Remove duplicate events by event_id.
2. Measure each device's clock offset (client minus server time) and how it changes.
3. Build a corrected event time: use client time adjusted by the device's offset, falling back to server time when the offset is unreliable.
4. Flag offline-download events and keep them, but label them separately.
5. Report how much the corrections change daily watch hours.

Expected output: Silver `playback_clean` with a corrected `event_ts`.

Concepts tested: deduplication, per-device statistics, `median`/`percentile_approx`, conditional correction, impact measurement.

Hint: Estimate each device's offset from its online events only (no `download_id`); a median is robust to the odd outlier. For offline events, server time is days late, so the corrected device time is the better clock.

Q3. Parse raw CDN access logs

Scenario: Two CDN providers send raw access-log text files, each in its own format. Every video segment request is one line. Finance wants bandwidth per title and region to check CDN invoices.

Sample line

```
2026-10-28T19:42:11Z edge-blr-03 a91f02c3 "GET /v/CNT00981/1080p/seg_00412.ts HTTP/2" 200 2387712 "StreamBharat-Android/7.4.1"
```

Data: cdn_logs from two providers.

Tasks

1. Parse both formats with regular expressions into: ts, cdn, edge, client_hash, content_id, quality, segment, status, bytes, user_agent.
2. Map edge locations to regions.
3. Quarantine lines that do not match the pattern, and count them per file.
4. Aggregate bytes per hour, CDN, region and content.

Expected output: Bronze `cdn_requests` and Silver `cdn_usage_hourly`.

Concepts tested: `spark.read.text`, `regexp_extract` with groups, multiple formats, quarantine, high-volume aggregation.

Hint: Write one regular expression per provider. The second provider uses epoch seconds and different edge names — `reference/cdn_edges.csv` maps both to regions.

Q4. Ingest player events incrementally

Scenario: Player events land in cloud storage every few minutes, about 4 billion a month. Re-reading everything daily is too slow and costly. The team wants an ingestion job that processes only new files, can run every hour or once a day, and never loses or duplicates events.

Data: playback_events folder with new files arriving continuously.

Tasks

1. Use Structured Streaming with a file source and `trigger(availableNow=True)` to process only new files, then stop.
2. Write to a Delta table partitioned by event date.
3. Show that running the job twice in a row processes nothing new the second time.
4. Handle a schema change: a new field `cast_device` appears in new files.

Expected output: an incremental Bronze ingestion job with checkpoints.

Concepts tested: file-source streaming as incremental batch, checkpoints, schema evolution with Delta `mergeSchema`.

Hint: File-source streaming needs an explicit schema and a checkpoint location. Look up how Delta handles a new column when you append (`mergeSchema`).

Q5. Build viewing sessions and watch time

Scenario: The player sends a heartbeat every 30 seconds while playing (every 5 minutes in the course dataset). The content team's key number is minutes actually watched, not minutes the app was open. Paused screens, skipped scenes and phones left playing overnight must not count.

Data: playback_clean, dim_content.

Tasks

1. For each device, order events and compute time between heartbeats; count only gaps of up to 6 minutes while playing (one heartbeat interval plus a margin).
2. Treat seeks forward as skipped content, not watched time.
3. Cap watch time per title at its duration multiplied by 1.5 (rewatching), and cut sessions with no interaction for 4 hours.
4. Build sessions (new session after 30 minutes of inactivity) with titles watched and minutes per title.
5. Compute completion: the share of a title's duration actually watched.

Expected output: `viewing_sessions` and `watch_time_daily` (profile, content, minutes, completion).

Concepts tested: `lag`-based interval sums, sessionisation, caps and business rules, completion ratios.

Hint: A long run of heartbeats with no pause, seek or stop for many hours is not a person watching. Positions that jump forward after a seek are skipped content.

Q6. Build the subscription lifecycle

Scenario: Finance and growth disagree on how many paying subscribers there are. Subscription events arrive from the billing service and app store gateways, sometimes twice, and a failed payment followed by a retry looks like a cancel and a new subscribe.

Data: subscription_events.

Tasks

1. Deduplicate events and order them per user.
2. Convert events into subscription periods: user, plan, status (trial, paid, paused, grace, churned), start_date, end_date.
3. Treat a payment failure recovered within 7 days as continuous, not a churn.
4. Count subscribers by plan and status on any given date.
5. Reconcile paid subscribers with revenue received from gateways.

Expected output: `subscription_periods` and `subscribers_daily`.

Concepts tested: event-to-state modelling, gaps-and-islands, grace-period rules, date spines, reconciliation.

Hint: Remove exact duplicates first. A payment failure followed by a renewal within 7 days is not a churn. `lead()` over each user's events gives the end of each period.

Q7. Clean search logs and find content gaps

Scenario: Users search in English, Hindi and transliterated Hindi ("pyaar", "pyar", "pyaar ka"). Searches with zero results show what users want that StreamBharat does not have, which is valuable for buying content.

Data: search_logs, dim_content.

Tasks

1. Normalise queries: lower case, trim, remove punctuation and repeated letters.
2. Group similar queries using `soundex` or `levenshtein` within the same first letter.
3. Compute search-to-play conversion: searches followed by a play within 10 minutes.
4. List the top 50 zero-result query groups per week and check whether any match a title name closely (a search problem, not missing content).

Expected output: `search_clean`, `search_conversion` and a weekly content-gap list.

Concepts tested: text normalisation, fuzzy grouping, time-window joins between logs, ranking.

Hint: Before calling a zero-result query a content gap, check its edit distance to existing titles — many are typos or spelling variants of titles you do have.

Q8. Track content tier and licence history

Scenario: A movie moved from premium to free in July, and its licence for one region ended in September. Reports on "premium watch time" and royalties must use the tier and licence valid when the viewing happened.

Data: catalog daily snapshots, a licences table (content_id, region, licence_start, licence_end).

Tasks

1. Build `dim_content_scd2` tracking tier, partner and title changes.
2. Join watch time to the tier valid at viewing time.
3. Flag viewing that happened in a region outside the licence period (a legal risk).
4. Show the catalog as it was on 1 August using Delta time travel.

Expected output: SCD2 content table and a licence-violation report.

Concepts tested: SCD2, point-in-time joins, range checks, Delta time travel.

Hint: Licences are by state, but viewing has a city — `reference/city_state.csv` bridges them. Compare the monthly catalog snapshots to find what changed and when.

Q9. Daily, weekly and monthly active users

Scenario: The board pack needs DAU, WAU and MAU, plus stickiness (DAU ÷ MAU), by plan, platform and state. The current numbers count a user twice when they use the phone and the TV.

Data: watch_time_daily, users, subscribers_daily.

Tasks

1. Define an active user as one who watched at least 1 minute that day; count distinct users, not devices.
2. Compute DAU per day, and WAU and MAU as rolling 7-day and 30-day distinct counts.
3. Compare exact and approximate distinct counts for speed and accuracy.
4. Break down by plan (free, basic, premium), platform and state; compute stickiness.

Expected output: `active_users_daily` with DAU, WAU, MAU and stickiness.

Concepts tested: rolling distinct counts, `approx_count_distinct`, multi-dimension aggregation.

Hint: Count a user on the day they actually watched (corrected device time from Q2), not the day the server received the event. Then compare your numbers with `reference/published_metrics.csv` — one month disagrees more than the others.

Q10. "Top 10 today" by region

Scenario: The home screen shows a "Top 10 in your state today" row. Editors complain it is often filled with one series' episodes or with titles that a few heavy viewers watched for hours.

Data: watch_time_daily, dim_content, users (state).

Tasks

1. Roll episodes up to their series.
2. Rank titles per state by distinct viewers who watched at least 2 minutes, using hours watched as a tie-breaker.
3. Exclude kids profiles from the general list and build a separate kids top 10.
4. Require a minimum of 50 viewers in that state, and fall back to the national list when a state has too little data.
5. Show each title's rank change compared with yesterday.

Expected output: `top10_daily` (date, state, rank, content, viewers, rank_change).

Concepts tested: rollups, `dense_rank` per group, fallback logic with joins, day-over-day comparison.

Hint: Episodes have their own content IDs; map them to their series with the catalog before ranking.

Questions 11–20

Q11. Subscriber cohorts and monthly revenue

Scenario: The CFO asks: "Of the people who subscribed in January, how many are still paying in June? And is our monthly recurring revenue (MRR) growing because of new subscribers or because fewer people cancel?"

Data: subscription_periods, subscribers_daily.

Tasks

1. Assign each subscriber to the cohort of their first paid month.
2. Build a retention table: cohort month × months since joining, with % still paying.
3. Compute MRR per month, and split the change into new, expansion (upgrades), contraction (downgrades), churned and reactivated MRR.
4. Compare retention of users acquired through different channels.

Expected output: `cohort_retention` (pivoted) and `mrr_bridge_monthly`.

Concepts tested: cohort assignment with `min` over window, `months_between`, `pivot`, period-over-period decomposition.

Hint: A subscriber's cohort is the month of their first paid event. For the MRR bridge, compare each user's monthly price with the previous month's.

Q12. Detect account sharing

Scenario: The basic plan allows 2 screens at a time, but some accounts stream on 6 devices across 4 cities. Management wants to know how common this is before deciding on a policy.

Data: playback_clean, viewing_sessions, subscription_periods.

Tasks

1. For each account, compute the maximum number of devices streaming at the same moment (overlapping sessions).
2. Count distinct devices and distinct cities per account per month.
3. Flag accounts that exceed their plan's screen limit on 5 or more days a month, or regularly stream from cities far apart at the same time.
4. Estimate how many households are behind flagged accounts and the possible revenue if each converted.

Expected output: `account_sharing_flags` and a revenue estimate.

Concepts tested: maximum concurrency from overlapping intervals, distinct counts, plan-limit lookups.

Hint: Put heartbeats into short time slots and count distinct devices per account per slot; compare with `max_screens` in `reference/plans.csv`. Offline playback should not count as concurrent streaming.

Q13. Advertising delivery and revenue

Scenario: Advertisers pay per 1,000 impressions (CPM) and set a frequency cap, for example "show my ad at most 3 times a day per user". An advertiser complains that some users saw their ad 12 times in one day while the campaign under-delivered overall.

Data: ad_impressions, ad_campaigns, viewing_sessions, dim_content.

Tasks

1. Compute fill rate: ad breaks that showed an ad ÷ total ad breaks served.
2. Compute impressions, completion rate and revenue per campaign per day.
3. Find frequency-cap violations per user, campaign and day, and the revenue that must be credited back.
4. Check genre targeting: impressions shown on content outside the campaign's target genres.
5. Report delivery against plan (budget ÷ campaign days) for each campaign.

Expected output: `ad_delivery_daily`, `frequency_violations` and a campaign pacing report.

Concepts tested: multi-join enrichment, array-membership checks, cap calculations, pacing measures.

Hint: Fill rate comes from `ad_breaks.csv`. A target of `ALL` means the campaign may run on any content.

Q14. Handle the scale of a live cricket final

Scenario: During the cricket final, the live match produced 60% of the day's playback events and CDN traffic. The nightly jobs that join events with content and compute watch time ran 5 times longer than usual because one content_id dominated.

Data: playback_clean, cdn_usage_hourly, dim_content, cdn_rates.

Tasks

1. Show the skew in the Spark UI and with key counts.
2. Fix it using AQE skew join handling, then try salting the live match's content_id, and compare.
3. Partition the event tables by event date and hour; Z-order by content_id; measure a single-title query before and after.
4. Compute CDN cost per title and region (bytes × rate per GB) for the day, and the cost of the final alone.

Expected output: a performance report and `cdn_cost_by_title`.

Concepts tested: skew detection and fixes, AQE, partitioning and Z-ordering, cost allocation.

Hint: Count events per content ID on 27 September before changing any settings. Salting adds a random bucket to the hot key on one side and repeats the other side once per bucket.

Q15. Pay content partners their royalties

Scenario: Content partners are paid a share of subscription revenue in proportion to minutes watched of their titles, with a minimum guarantee per quarter. Free-tier viewing is paid from ad revenue instead. A partner disputes last quarter's payout.

Data: watch_time_daily (premium viewing), dim_content_scd2 (partner at viewing time), subscription revenue per month, ad revenue per title (Q13), content_partners.

Tasks

1. Compute each partner's share of premium minutes per month, using the partner valid at viewing time.
2. Allocate the partner pool (subscription revenue × revenue-share %) by that share.
3. Add each partner's ad revenue share from free-tier viewing of their titles.
4. Apply the quarterly minimum guarantee: pay the higher of earned royalty and guarantee.
5. Make sure the total paid equals the pool plus top-ups, to the paisa, and produce a per-title statement for the disputing partner.

Expected output: `partner_royalties_quarterly` and per-partner statements.

Concepts tested: proportional allocation, rounding so totals still match, guarantee logic, audit-ready outputs.

Hint: Use the partner valid at viewing time (some titles changed partner). After allocating and rounding, check the total paid still equals the pool — decide where the leftover paise go.

Q16. Build recommendations with collaborative filtering

Scenario: The "Because you watched…" row is currently filled by genre only, and users rarely click it. The product team wants recommendations based on what similar viewers watched.

Data: watch_time_daily, ratings, dim_content.

Tasks

1. Build an implicit feedback score per profile and title from completion and rewatching (explicit ratings where available).
2. Map string IDs to integers, and train an ALS model in MLlib with `implicitPrefs=True`.
3. Hold out each profile's last 5 titles and measure precision at 10.
4. Remove titles already watched, titles not licensed in the user's region, and adult titles for kids profiles.
5. Compare against a simple "most popular in your language" baseline.

Expected output: `recommendations` (profile, 10 titles) and an evaluation report.

Concepts tested: implicit feedback, `StringIndexer`, ALS, ranking evaluation, post-filtering with business rules.

Hint: ALS needs integer IDs. Read about `implicitPrefs` and `coldStartStrategy` in the MLlib documentation.

Q17. Real-time "Trending now"

Scenario: When a new trailer or episode drops, the "Trending now" row should update within minutes, not the next day. Trending means rising fast compared with normal, not simply most watched.

Data: a live stream of playback start events, dim_content.

Tasks

1. Count distinct viewers per title in 10-minute windows sliding every minute, with a watermark.
2. Compare each title's current count with its average for the same hour over the last 7 days (stored in a table).
3. Rank titles by growth ratio, with a minimum of 200 viewers, and output the top 20 every minute.
4. Keep a separate list per language.

Expected output: `trending_now` stream table.

Concepts tested: sliding windows, approximate distinct counts in streams, stream-static joins with baselines, top-N per window.

Hint: Exact distinct counts are not supported in streaming aggregations — look for the approximate version. Build the 7-day baseline in batch from the playback history.

Q18. Live ad pacing and frequency capping

Scenario: During the live match, ad breaks happen every over. Some campaigns spend their entire daily budget in the first hour, and the Q13 frequency-cap problem must be prevented live, not credited back later.

Data: a live stream of ad impressions, ad_campaigns.

Tasks

1. Keep a running count of impressions per user, campaign and day in state.
2. Output users who have reached the cap so the ad server can stop showing them that ad.
3. Track spend per campaign versus its ideal pace (budget spread across the match hours); alert when a campaign is over 120% or under 70% of pace.
4. Reset counters at midnight.

Expected output: `capped_users_live` and `pacing_alerts` stream tables.

Concepts tested: stateful aggregation, state reset by date, pacing calculations, multiple outputs from one stream.

Hint: Keep counts per user, campaign and date; putting the date in the key is the simplest way to reset at midnight.

Q19. Data quality, kids-safety checks and tests

Scenario: A parent complained that an adult-rated title played on a kids profile. Separately, the board noticed that last month's DAU figure changed after it was published.

Data: playback_clean, dim_content_scd2 (with age rating), profiles, all Gold tables.

Tasks

1. Find every play of a title rated 16+ or 18+ on a kids profile, and the app version used.

2. Add quality gates: watch minutes per profile per day cannot exceed 24 hours, every play links to a known title, royalties total equals the pool.
3. Freeze published monthly numbers: store each published value and alert if a recalculation changes it by more than 0.5%.
4. Write `pytest` tests for watch-time calculation (Q5), subscription periods (Q6) and royalty allocation (Q15).

Expected output: a kids-safety report, quality results, a published-metrics log and passing tests.

Concepts tested: compliance queries, quality gates, metric versioning, unit tests.

Hint: Join playback with profiles (`is_kids`) and the catalog's `age_rating`, then group the violations by app version — the pattern tells you what broke.

Q20. Subscriber churn model and the full pipeline

Scenario: About 9% of paying subscribers cancel each month. The retention team can offer a discount to 50,000 subscribers before renewal. They want the ones most likely to leave and still worth saving. The platform team wants the StreamBharat pipeline to run nightly with one command.

Data: subscription_periods, watch_time_daily, viewing_sessions, search_conversion, payment failures, account_sharing_flags.

Tasks

1. Define churn as not renewing within 7 days after the period ends; build features only from data before the renewal date.
2. Features: watch minutes trend over the last 4 weeks, days active, titles started vs finished, failed searches, payment failures, plan, tenure, language preference.
3. Train an MLlib classifier with a time-based split; report AUC and the top features.
4. Score subscribers renewing next week; rank by churn risk × plan value; output the top 50,000.
5. Write `pipeline.py` that runs ingestion → Silver → quality gates → Gold → scoring with logging and failure codes.

Expected output: model report, a weekly retention list and a one-command pipeline.

Concepts tested: leakage-free labels, trend features with windows, MLlib classification, value-based ranking, orchestration.

Hint: Involuntary churn (payment failure) and voluntary churn behave very differently before the event; check watch minutes in the weeks before each kind.